

Министерство образования Новосибирской области
ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б.С. Галуцака»

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

Автоматизация сборки и деплоя проекта через GitHub Actions/GitLab и Docker
Методические указания к практическому занятию №15

Учебная дисциплина: ОПВ.15 Проектирование, разработка и оптимизация веб-приложений
Специальность: 09.02.07 Информационные системы и программирование

Разработал:
Зиберева А.А.

1 Цели

1.1 В ходе выполнения работы студенты осваивают:

1.1.1 Общие компетенции:

ОК 01 — Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам

ОК 02 — Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

ОК 09 — Пользоваться профессиональной документацией на государственном и иностранном языках

1.1.2 Профессиональные компетенции:

ПК 9.2 — Разрабатывать веб-приложение в соответствии с техническим заданием

ПК 9.6 — Размещать веб приложения в сети в соответствии с техническим заданием

1.2 В результате выполнения практического задания студенты:

1.2.1 Приобретают практический опыт:

- подготовки программного проекта к контейнеризации с использованием Docker;
- создания и настройки файла Dockerfile для автоматической сборки Docker-образа;
- создания файла .dockerignore для исключения ненужных и конфиденциальных файлов из контекста сборки;
- сборки Docker-образа проекта и его запуска в виде контейнера;
- настройки взаимодействия локального проекта с репозиторием GitHub;
- создания и настройки workflow GitHub Actions для автоматизации процессов тестирования и сборки проекта;
- автоматической сборки Docker-образа после внесения изменений в репозиторий;
- публикации собранного Docker-образа в GitHub Container Registry (GHCR);
- анализа результатов выполнения заданий GitHub Actions и журналов выполнения (логов).

1.2.2 Осваивают навыки:

- разработки Dockerfile с использованием инструкций FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, CMD;
- настройки .dockerignore с учетом структуры и требований проекта;
- выполнения команд docker build, docker run, docker ps, docker stop для управления Docker-образами и контейнерами;
- создания репозитория на GitHub и размещения в нем исходного кода проекта;
- работы с системой контроля версий Git для фиксации и отправки изменений в удаленный репозиторий;
- создания файлов конфигурации GitHub Actions в каталоге .github/workflows/;
- настройки триггеров, задач, зависимостей и условий выполнения workflow;
- использования готовых Actions для получения исходного кода, настройки среды выполнения и сборки Docker-образов;
- настройки автоматической публикации Docker-образов в GitHub Container Registry;
- поиска и анализа ошибок по журналам выполнения GitHub Actions;
- безопасного хранения ключей, паролей и других конфиденциальных данных с использованием Secrets и Variables GitHub.

1.2.3 Усваивают знания:

- основных принципов контейнеризации программных приложений с использованием Docker;
- назначения и структуры Dockerfile, а также принципов формирования Docker-образа;
- назначения файла .dockerignore и принципов исключения ненужных и конфиденциальных данных из контекста сборки;
- принципов работы Docker-образов и контейнеров, а также их взаимодействия с операционной системой и сетевыми портами;

- основных принципов непрерывной интеграции и непрерывной доставки/развертывания (CI/CD);
- назначения и структуры workflow GitHub Actions, а также принципов работы triggers, jobs, steps, runners и Actions;
- принципов автоматизации тестирования и сборки программного проекта;
- назначения GitHub Container Registry и принципов хранения в нем Docker-образов;
- принципов взаимодействия системы контроля версий Git, GitHub и GitHub Actions;
- способов безопасного хранения секретных данных, необходимых для выполнения автоматизированных задач;
- основных различий между GitLab CI/CD и GitHub Actions при организации автоматизированных процессов разработки и сборки

2 Оборудование, аппаратура, материалы, инструменты, документация, дидактическое обеспечение

- персональный компьютер с установленной операционной системой Windows, macOS или Linux;
- доступ к сети Интернет;
- веб-браузер Google Chrome, Mozilla Firefox или аналогичный;
- интегрированная среда разработки (IDE) Visual Studio Code или аналогичная;
- установленная система контроля версий Git;
- установленный Docker Desktop;
- учетная запись GitHub с возможностью создания и использования репозитория;
- исходный код индивидуального проекта студента;
- файл Dockerfile, настроенный в соответствии со структурой проекта;
- файл .dockerignore;
- файл конфигурации workflow GitHub Actions в каталоге .github/workflows/;
- доступ к GitHub Container Registry (GHCR) для хранения Docker-образов;
- документация по Docker и GitHub Actions;
- методические указания к практическому занятию;
- инструкции и примеры выполнения заданий, приведенные в приложении А;
- программные средства и зависимости, необходимые для сборки и тестирования конкретного проекта.

3 Форма организации – индивидуальная (групповая в случае нехватки рабочих мест).

4 Инструктаж

4.1 Время выполнения работы – 6 академических часов (270 минут)

4.2 Работа состоит из заданий, приведенных ниже в пункте 9

4.3 При выполнении работы следует пользоваться методическими указаниями, приведенными в приложении А, либо конспектом соответствующей темы и выполнить задания

5 Порядок выполнения

5.1 Ознакомиться с теоретическими сведениями о практической работе

5.2 Ознакомиться с формой отчета по практическому занятию

5.3 Ознакомиться с методическими указаниями, приведенными в приложении А

5.4 Выполнить работу, оформить отчет, предоставить оба на проверку преподавателю

6 Методические рекомендации

6.1 Методические рекомендации приведены в приложении А

7 Форма отчета

Отчет должен содержать:

- номер и название практического занятия;
- фамилию, номер группы и вариант студента;
- выполненные задания практической работы.

8 Критерии оценок

- «Отлично» - при условии выполнения всех заданий;
- «Хорошо» - при условии выполнения всех заданий, но допущены неточности в их решении;
- «Удовлетворительно» - допущены ошибки при выполнении заданий, отчет оформлен частично,
- «Неудовлетворительно» - если практические задания не выполнены или допущены ошибки, влияющие на качество выполненной работы, отчет по работе отсутствует.

9 Содержание заданий

Задание

Необходимо выполнить практическое задание, направленное на контейнеризацию программного проекта с использованием Docker и автоматизацию его тестирования, сборки и публикации Docker-образа посредством GitHub Actions.

При выполнении работы используйте документацию и сеть Интернет для адаптации инструкций к своему проекту.

ПРИЛОЖЕНИЕ А

Инструкции к выполнению заданий

Теоретический материал по теме практической работы

1 Основные понятия для понимания GitLab и DevOps (и не только)

В другой работе у вас уже был материал по изучению Docker. К этому материалу все также можно вернуться и уточнить свои знания позже. Теперь перейдем к изучению GitLab.

DevOps — это методология и корпоративная культура, объединяющая разработку программного обеспечения (Dev) и ИТ-операции/эксплуатацию (Ops) для ускорения и повышения надежности выпуска продуктов. Главными принципами являются: сотрудничество между разными людьми при разработке продукта, минимизация ручного труда, и постоянная интеграция кода и его быстрая доставка до конечного пользователя. Т.е. когда раньше программисты и условные тестировщики с сисадминами были все «разделены», сейчас — появляются новые инструменты, которые позволяют всем участникам разработки иметь общую ответственность, автоматизацию всех процессов разработки и запуска непрерывного конвейера для разработок, тестирования и запуска продуктов и их новых функций и возможностей. Данная философия отказывается от перебрасывания кода туда-сюда как «горячей картошки» для того, чтобы оптимизировать процесс цикла жизни программного продукта.

Конечно, все инструменты данной методологии мы не сможем рассмотреть, но GitLab — это единая DevOps-платформа по принципу «всё в одном» (All-in-One), которая покрывает весь жизненный цикл разработки ПО. В методологии DevOps её используют практически на каждом этапе.

GitLab — это веб-платформа для управления репозиториями кода Git и совместной разработки программного обеспечения, охватывающая весь жизненный цикл DevOps (планирование, написание, тестирование и развертывание).

Главное отличие GitLab от GitHub заключается в том, что GitLab создан как единая интегрированная платформа для всего цикла DevOps, а GitHub — это популярный облачный хостинг и социальная сеть для разработчиков с упором на открытый исходный код.

Особенности GitLab состоят в следующем:

- планирование и управление — организация кода и планирование задач;
- создание и разработка — классическое управление исходным кодом (SCM) на базе Git;
- интеграция (непрерывная, CI) — проверка кода, автоматический запуск тестов;
- упаковка и хранение — управление контейнерами и зависимостями;
- безопасность — интеграция проверок в процесс разработки (DevSecOps — методология создания программного обеспечения, которая объединяет разработку, эксплуатацию и информационную безопасность в единый непрерывный процесс);
- непрерывное развертывание — деплой и управление доступностью функций;
- мониторинг и защита — метрики и другие полезные штуки для поддержки ПО.

GitLab Runner — это специальный агент (программное обеспечение), который выполняет задачи из CI/CD-пайплайнов (автоматических конвейеров сборки, тестирования и деплоя) в GitLab. Без этих небольших приложений автоматизация работать не будет. Как работают «раннеры»:

- сервер GitLab формирует задание, когда разработчик отправляет код (push) или создает запрос на слияние (merge request). Раннер постоянно опрашивает GitLab через API, забирает свободную задачу в работу;
- агент читает конфигурационный файл `.gitlab-ci.yml` из репозитория и по шагам выполняет прописанные там команды (например, сборка проекта, запуск тестов, выгрузка на сервер);
- после выполнения задач раннер отправляет логи и итоговый статус (успех или ошибка) обратно в веб-интерфейс GitLab.

Или же проще:

- при push или создании Merge Request сервер GitLab формирует задание;
- раннер забирает задание и читает файл `.gitlab-ci.yml`;
- выполняет прописанные команды (сборка, тесты, деплой и т. д.);

- отправляет логи и статус (успех/ошибка) обратно в GitLab.

Раннеры бывают разных видов: **shared** (общие) — доступны для всей группы проектов или всего экземпляра GitLab; **specific** (специальные) — привязаны к конкретному проекту или группе; **group** (групповые) — общие для всех проектов, находящихся внутри конкретной группы.

Раннер может запускать команды в разных средах, например Shell, Docker, Kubernetes, VirtualBox.

На рисунке 1.1 можно увидеть примерную схему работы данной системы.

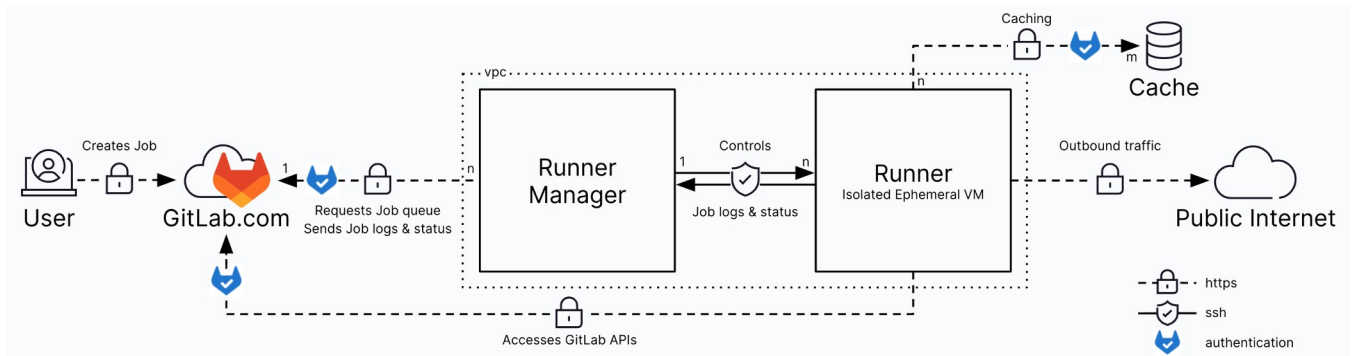


Рисунок 1.1 — Архитектурная схема размещенных раннеров для GitLab.com

CI/CD-пайплайн (конвейер непрерывной интеграции и доставки) — это автоматизированная цепочка шагов и задач, которая доставляет изменения в коде от компьютера разработчика до реальных пользователей.

Из чего состоит CI/CD:

- CI (Continuous Integration/непрерывная интеграция): разработчики часто объединяют код в общий репозиторий. Система автоматически собирает проект и запускает тесты, чтобы найти ошибки на ранних этапах;

- CD (Continuous Delivery/Continuous Deployment — непрерывная доставка или развертывание): проверенный код автоматически или по одной кнопке отправляется на тестовый или рабочий сервер (продакшен).

GitLab-пайплайн — это автоматический сценарий, который проверяет, собирает и выкатывает ваш код.

Конвейер (pipeline) в GitLab состоит из нескольких ключевых компонентов, которые совместно определяют, запускают и выполняют ваш рабочий процесс CI/CD:

- `.gitlab-ci.yml` — файл конфигурации конвейера, расположенный в корне (или по указанному пути) репозитория. В нем описываются задачи (jobs), этапы (stages), переменные, триггеры и многое другое;

- этапы (Stages) — упорядоченные фазы (например, сборка, тестирование, развертывание), которые проходят задачи. Задачи одного этапа выполняются параллельно, а задачи разных этапов — последовательно;

- задачи (Jobs) — отдельные единицы работы, включающие этап, скрипт, образ, артефакты, зависимости (needs), правила (rules) и другие настройки. Каждая задача имеет: Runner — исполнитель (управляемый GitLab или собственный), который выполняет задачу; артефакты (Artifacts) — файлы, создаваемые в ходе выполнения и передаваемые между задачами; кэш (Cache) — данные, сохраняемые для повторного использования между запусками; зависимости (Dependencies & needs) — механизмы управления порядком выполнения задач и их параллельностью;

- переменные (Variables) — конфигурационные данные конвейера (переменные проекта, переменные CI/CD, переменные окружения, маски, области видимости/защищенные переменные);

- триггеры (Triggers) — события, запускающие конвейер (push, merge request, расписание, вебхук, API, ручной запуск или запуск по триггеру);

- среды (Environments) — именованные цели развертывания с URL-адресами и настройками режима просмотра (review apps);

- правила (Rules/only/except) — условия, определяющие, будет ли выполняться задача;

- интерфейс конвейеров (Pipelines UI) — часть интерфейса GitLab для просмотра запусков, логов и артефактов, а также управления задачами (повторный запуск, отмена, ручное подтверждение).

На рисунке 1.2 изображен пример пайплайна. Здесь, к примеру, прописано правило на стадии продакшена для ручного действия, поэтому оно не происходит автоматически.

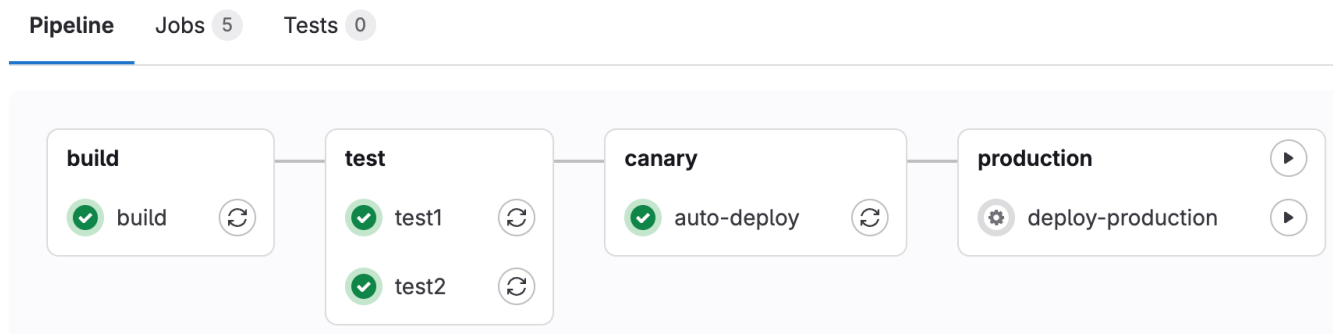


Рисунок 1.2 — Работа пайплайна с ручным действием на стадии продакшена

Как мы можем видеть, здесь есть почти все основные составные части: конвейер, задачи, тесты и стадии. Также, задачи находятся внутри каждой стадии и отмечаются разными индикаторами. Здесь они по большей части выполнены.

Если же какая-то из задач не получится (failed), то конвейер (прямо как на заводе при какой-то ошибке) остановится полностью и сообщит о том, на каком этапе произошла ошибка. На рисунке 1.3 показан пример.

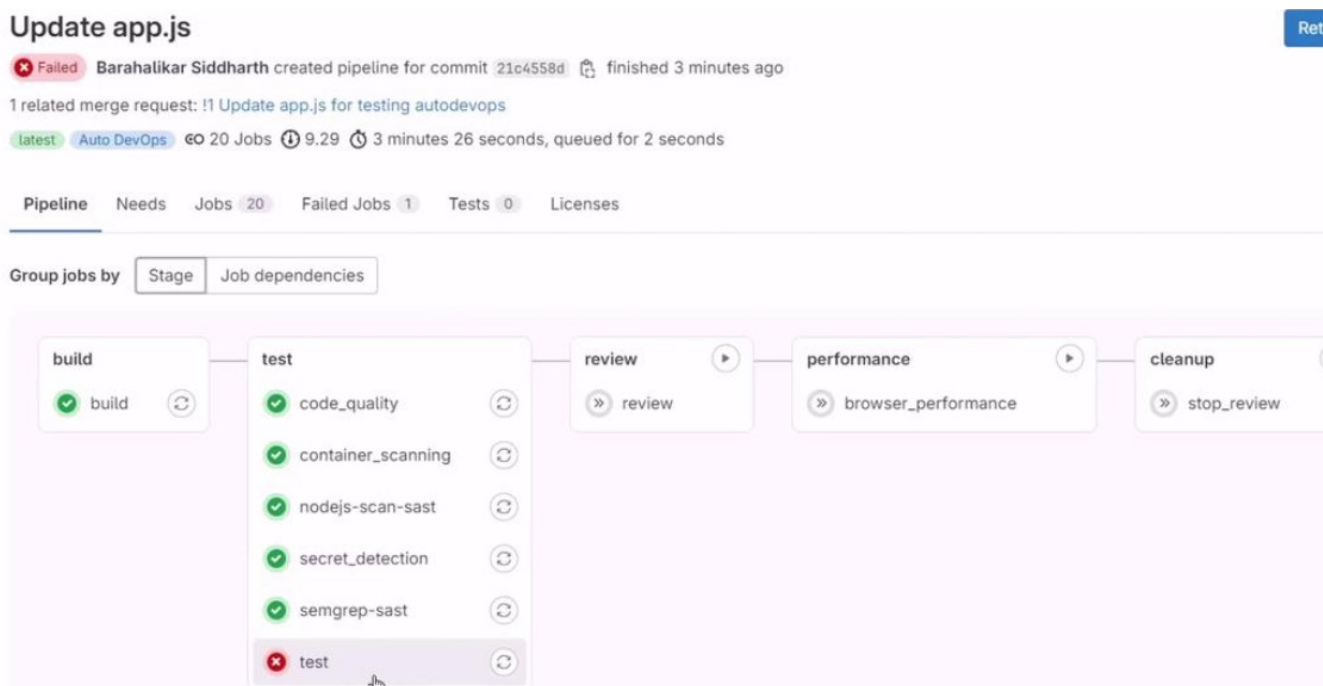


Рисунок 1.3 — Пайплайн с провалившейся работой/задачей

Так как мы уже немного знакомы с Docker, нужно понимать, что чтобы с ним работать, нужно отдельно его подключать, но не в случае GitLab, так как там присутствует встроенный Container Registry — это встроенный в GitLab репозиторий Docker-образов. Он позволяет хранить собранные Docker-образы для каждого проекта прямо внутри GitLab, не используя сторонние сервисы (вроде Docker Hub или Nexus). Вы можете собрать Docker-образ прямо внутри CI/CD-конвейера и тут же «сложить» его в репозиторий этого же проекта. Это невероятно удобно для соблюдения принципа «всё в одном».

Container Registry нужен для безопасного хранения, управления и быстрого распространения образов контейнеров (например, Docker), которые используются для запуска приложений. Без него разработчикам и серверам было бы негде брать готовые сборки программ.

GitLab Merge Request (MR) — это механизм запроса на слияние изменений из одной ветки кода в другую. Этот инструмент служит центральным местом для командной работы, обсуждения кода, проведения код-ревью и автоматических проверок перед тем, как изменения попадут в основную ветку.

Для чего может использоваться данная функция:

- код-ревью: коллеги по проекту могут прокомментировать код, оставить свои предложения;
- обсуждения: все вопросы по реализации задачи решаются прямо на странице запроса;
- автоматические тесты и проверки перед слиянием: система запускает тесты и проверки, чтобы убедиться, что новый код ничего не сломал;
- история изменений.

Изменения создаются в отдельной рабочей ветке, а MR запрашивает их перенос (слияние) в базовую ветку (например, `main` или `develop`), после чего новая ветка исчезает.

Также, GitLab предоставляет функцию хостинга, которая называется GitLab Pages — это бесплатный хостинг для статических сайтов, который позволяет публиковать веб-страницы прямо из репозитория на платформе GitLab. Также, GitLab помогает хостить проекты на Vercel благодаря автоматизации процесса развертывания (CI/CD) и синхронизации кода. Принцип все тот же — обновление контента автоматическое/по вашим инструкциям. Для MR на Vercel делается отдельная ветка, чтобы протестировать код до того, как его слить с главной веткой.

Рассмотрим важные моменты далее. Как мы с вами уже знаем, что все переменные окружения (ключи, пароли и т.д.) ни в коем случае нельзя хранить в репозитории Git. Специально для этого мы делали файл `.gitignore`, который исключал такие файлы и деревья коммитов. Здесь в игру вступает возможность хранить переменные прямо в проекте GitLab.

GitLab variables (переменные CI/CD в GitLab) — это специальные переменные окружения, которые хранят нужные данные и передаются в автоматизированные задачи (работы) при сборке и тестировании кода. Вместо того чтобы «зашивать» (hardcode) секретный ключ от базы данных прямо в код или файл `.gitlab-ci.yml`, вы сохраняете его в Variables через веб-интерфейс GitLab, и он становится доступен в процессе сборки как обычная переменная окружения.

Однако, чтобы всё было максимально защищено, при добавлении ключа в GitLab обязательно используйте две важные настройки:

- Masked (Маскирование): активируйте этот переключатель, тогда GitLab автоматически заменит ваш ключ на звездочки в логах сборки;
- Protected (Защита): если включить эту опцию, переменная будет передаваться только в те ветки и теги, которые отмечены как защищенные (например, `main` или `master`).

GitLab использовался для примера того, как это может быть и как это может работать. Для самой работы будет использоваться GitHub Actions, которая очень похожа на GitLab за некоторыми исключениями. GitLab не дает Российским пользователям использовать пайплайны без верификации через иностранный номер телефона.

2 Основные понятия для понимания GitHub Actions

GitHub Actions — это встроенная в GitHub платформа для автоматизации процессов разработки, тестирования и развертывания программного обеспечения (CI/CD). Она отвечает всем тем же требованиям, которые есть у GitLab — такие же некоторые функции и возможности, почти похожий интерфейс, те же цели, достигаемые при помощи инструмента. Данная вкладка находится в каждом репозитории автоматически, как показано на рисунке 2.1.

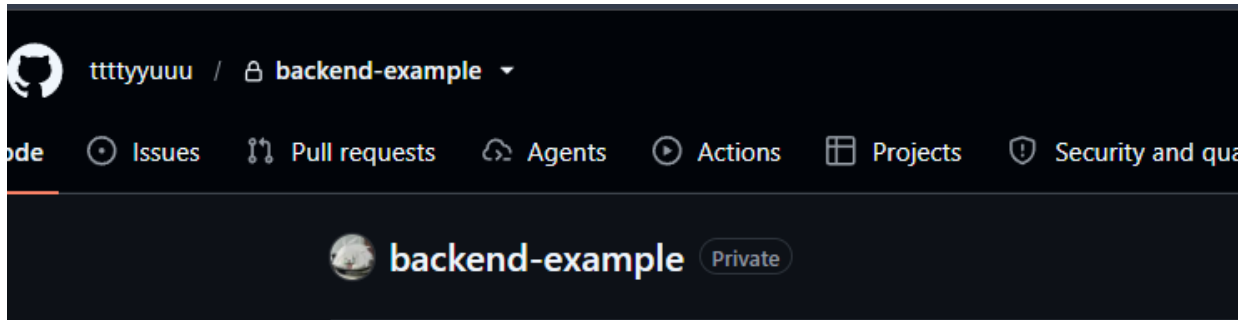


Рисунок 2.1 — Вкладка Actions на верхней панели управления репозиторием

Главное отличие заключается в том, что GitHub Actions — это часть экосистемы GitHub, а GitLab CI/CD — часть единого монолитного приложения GitLab, где всё (от репозитория и CI/CD до трекера задач и мониторинга) находится "из коробки" в одном интерфейсе.

Основные различия:

- у GitHub Actions есть огромный GitHub Marketplace, где разработчики со всего мира делятся готовыми шагами и скриптами (actions); в GitLab готовые шаблоны встроены внутри системы или подключаются через включение внешних файлов, но сообщество готовых мелких экшенов меньше;
- GitHub Actions описывает шаги выполнения через ключ `run` (для команд) и `uses` (для готовых модулей), а файлы конфигурации хранятся в папке `.github/workflows/` (а как это делается в GitLab — мы уже знаем);
- GitHub Actions работает преимущественно в облаке GitHub (хотя можно подключить и свои серверы — self-hosted runners);
- GitLab позволяет настраивать правила хранения артефактов и кэша более гибко и явно в одном конфигурационном файле; GitHub Actions требует прописывать отдельные шаги для загрузки и скачивания файлов через специальные действия (actions/cache или actions/upload-artifact).

Конечно, есть также отличие в том, что GitHub — модульный проект, включающий в себя не только знакомую нам систему контроля версий и GitHub Actions, а также GitHub Projects (вкладка Projects) (для сложных задач часто приходится подключать сторонние Jira или Trello); используется GitHub Packages (он тоже встроен, но настраивается через Actions как внешнее хранилище, куда нужно отдельно логиниться). Но для небольшого понимания данного процесса данной платформы практически достаточно.

Большую часть того, что мы увидели в GitLab мы найдем в GitHub и воспользуемся этими функциями.

Ознакомиться с документацией GitHub Actions и другими возможностями можно по [гиперссылке](#).

На картинке ниже примерно показано, как работает вся эта система. Обратите внимание на рисунок 2.2.

GitHub Actions Components

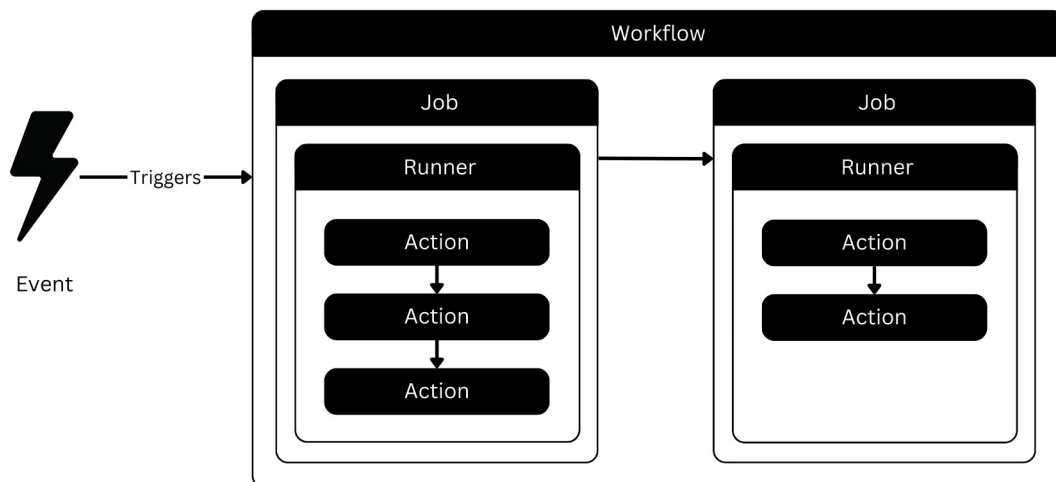


Рисунок 2.2 — Компоненты GitHub Actions

Так как у вас уже есть аккаунт в GitHub, для нас начать эту работу не составит никакого труда.

Для дальнейшего понимания того, с чем мы будем работать, необходимо перейти непосредственно к шагам работы с нашими инструментами. Для примера в методических рекомендациях будет использоваться небольшой API на Flask в связке с Firebase.

Проектами, которые вы будете использовать во время данной работы, должны стать ваши начальные наработки по самому программному продукту индивидуального проекта, в зависимости от того, как и с чего именно вы начали его разрабатывать.

Шаги выполнения работы

1 Пример проекта

Как было сказано ранее, для примера будет использоваться небольшой проект, написанный на Python (с использованием Flask), который является API для связи с Firebase. В нем есть несколько запросов на создание, удаление, изменение и нахождение товара в базе данных (примерно похожий проект уже был в одной из практических работ по разработке веб-приложения с бэкендом как отдельным проектом).

Ваши проекты могут быть любыми, в зависимости от того, какие технологии и какой подход вы выбрали для разработки: может, у вас нет отдельного backend и frontend, может вы использовали Node.js вместо Python — возможности разные, просто некоторые моменты необходимо адаптировать под ваш программный продукт, хотя принцип примерно один и тот же.

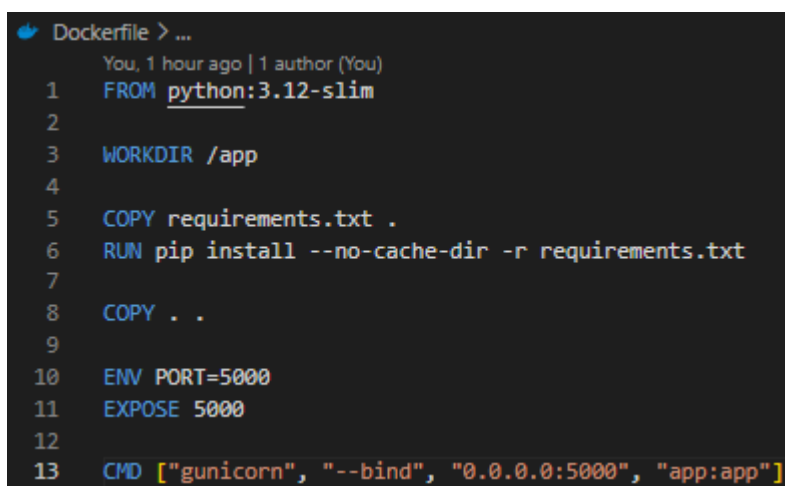
Также, проект из примера имеет небольшую документацию Swagger, встроенную в запросы.

Начнем с настройки и запуска контейнера (или контейнеров, если у вас две или больше составных частей проекта).

2 Dockerfile, .dockerignore и Docker Desktop

Напомню, что Docker можно пользоваться и не только с помощью десктопной версии, но здесь будет использоваться именно она, так как это гораздо удобнее.

В корне проекта нам необходимо написать Dockerfile. У него нет расширения, только название с большой буквы. Ваша среда разработки сама узнает данный файл и, возможно, подскажет какие расширения доустановить, чтобы с ним работать. Теперь необходимо его заполнить. Повторюсь, что мои файлы — только примеры. В сети Интернет вам необходимо найти как именно сделать конфигурацию, подходящую под ваш проект. Посмотрим на пример файла на рисунке 2.1.



```
Dockerfile > ...
You, 1 hour ago | 1 author (You)
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY . .
9
10 ENV PORT=5000
11 EXPOSE 5000
12
13 CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

Рисунок 2.1 — Пример Dockerfile

Напомню состав файла:

- FROM — задаёт базовый образ, с которого начинается сборка (например, официальный образ Node.js, Python или Ubuntu); все существующие образы можно найти на Docker Hub, а здесь используется slim-версия для Python; slim-образы для Docker нужны для уменьшения размера контейнеров и повышения безопасности;

- WORKDIR — устанавливает рабочую директорию (папку) внутри контейнера для всех последующих команд (COPY, RUN, CMD). Если папки нет, она создаётся;

- COPY — копирует файлы или папки с вашего компьютера (хоста) внутрь контейнера; здесь мы копируем требования (библиотеки);

- RUN — выполняет консольную команду (например, установку зависимостей npm install или pip install) во время сборки образа. Создаёт новый слой;

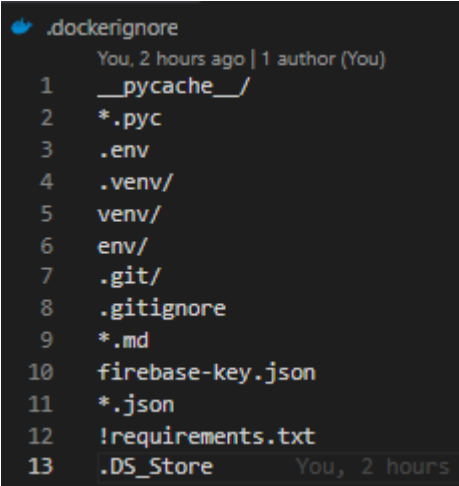
- ENV PORT= — задаёт постоянные переменные окружения (например, порт приложения);

- EXPOSE — документирует, на каком порту приложение будет слушать сетевые запросы (но не открывает порт автоматически наружу);

- CMD [""] — указывает команду по умолчанию, которая запускается при старте контейнера.

Т.е. как и было сказано в другом материале по Docker – Dockerfile — это текстовый файл без расширения, который содержит набор инструкций и команд для автоматической сборки Docker-образа. Каждая строка в файле задает отдельный шаг (например, установить операционную систему, скопировать файлы, запустить программу), при выполнении команды сборки Docker читает файл построчно и создает неизменяемые слои файловой системы, а полученный образ гарантирует, что приложение запустится с одинаковым окружением на любом компьютере.

Далее, перейдем к файлу .dockerignore. Это текстовый файл, который указывает Docker, какие файлы и папки нужно исключить из контекста сборки. На рисунке 2.2 рассмотрим пример такого файла (у вас он будет отличаться).



```
.dockerignore
You, 2 hours ago | 1 author (You)
1  __pycache__/
2  *.pyc
3  .env
4  .venv/
5  venv/
6  env/
7  .git/
8  .gitignore
9  *.md
10 firebase-key.json
11 *.json
12 !requirements.txt
13 .DS_Store
You, 2 hours
```

Рисунок 2.2 — .dockerignore

Его главная цель — запретить системе контроля версий (Git) или Docker копировать лишние файлы. Сюда входят кэш, системный мусор, папки виртуального окружения и приватные ключи, которые не должны попасть в публичный репозиторий (например, на GitHub) или внутрь Docker-контейнера.

Сюда входят: кэш и файлы компиляции, файлы окружения, файлы Git, документация, приватные ключи, но стоит восклицательный знак, для того, чтобы исключить файл requirements.txt — восклицательный знак означает «инвертировать правило» (не игнорировать); то есть, несмотря на то, что строчкой выше мы запретили файлы *.json (а в некоторых конфигурациях запрещают и *.txt), файл requirements.txt Git/Docker обязан заметить и сохранить, так как в нем записаны все зависимости проекта.

Теперь наш образ необходимо собрать. Для этого воспользуемся командой docker build -t backend-example . — где два слова перед точкой — название проекта.

На рисунке 2.3 представлен результат данной команды в терминале.

```
(env) PS D:\projects\backend-example> docker build -t backend-example .
[+] Building 39.3s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 254B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 170B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> [internal] load build context
=> => transferring context: 4.23kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:fd1ac6b4319ca3c53e2a77322785b94244166fc616e0bdfc38d610c74ffc8461
=> => exporting config sha256:b9706088cf2e87539b7b467728f98cbabaa804a188e814ff27adeb2323a7930f
=> => exporting attestation manifest sha256:aa327b5f695a72eba71a097a8f911f7a2a4a91a477d8ddf72f6ff97afd4b5fd
=> => exporting manifest list sha256:3fb6c8e1c3d5bdc4a8097d96095e36742763f213e6a1517298bca456fcd042fb
=> => naming to docker.io/library/backend-example:latest
=> => unpacking to docker.io/library/backend-example:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/k6p0ctilvc6xwrid3gt688ygl
```

Рисунок 2.3 — Пример сборки образа Docker

Docker взял чистую операционную систему, зашел внутрь нее, перенес туда файлы вашего проекта с компьютера, установил нужные библиотеки и «запечатал» всё это в готовый слепок (образ), который теперь называется `backend-example:latest`. Произошло это в несколько этапов:

- подготовка;
- скачивание основы (образ);
- создание рабочей папки;
- установка библиотек;
- копирование кода;
- запечатывание.

Образ полностью готов. Теперь его можно запустить командой:

`docker run -p 5000:5000 -e GOOGLE_APPLICATION_CREDENTIALS=/app/firebase-key.json -v ${PWD}/firebase-key.json:/app/firebase-key.json backend-example` — разберем ее составные части.

Общими словами, мы запускаем контейнер, прокидываем внутрь него сетевой порт, передаем переменную окружения и подключаем файл ключа Firebase:

- `-p 5000:5000` — связывает порт вашего компьютера с портом внутри контейнера; первая 5000 (слева) — это порт на реальном компьютере (хосте); вторая 5000 (справа) — это порт, который слушает Python-приложение внутри контейнера;
- `-e GOOGLE_APPLICATION_CREDENTIALS=/app/firebase-key.json` — создает внутри контейнера переменную среды (флаг `-e` от слова Environment); библиотеки Google и Firebase автоматически ищут эту переменную, чтобы понять, где лежит ключ авторизации;
- `${PWD}/firebase-key.json` (слева от двоеточия) — путь к файлу на настоящем компьютере; `${PWD}` автоматически подставляет путь к текущей папке, в которой мы сейчас находимся в терминале;
- `/app/firebase-key.json` (справа от двоеточия) — это то, где этот файл виртуально появится внутри контейнера.

После того как мы его запустили, можно перейти в Docker Desktop и увидеть наш контейнер. На рисунке 2.4 показано, как это выглядит.

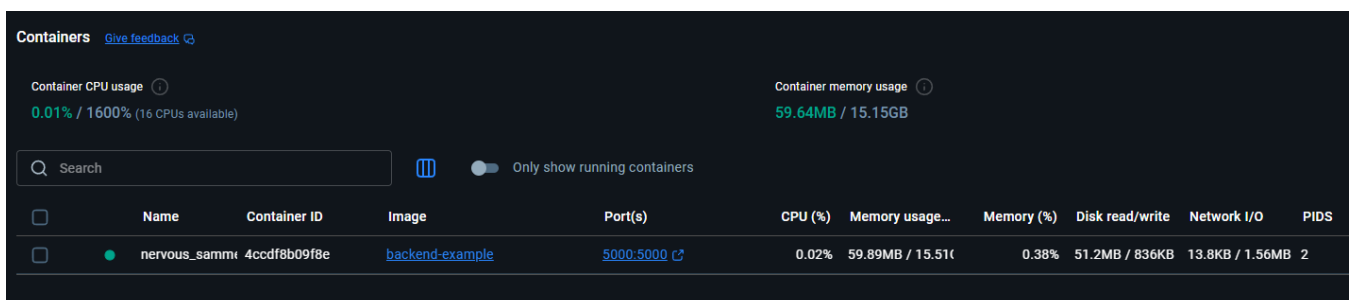


Рисунок 2.4 — Пример запущенного контейнера

Также, можно увидеть образы, как это показано на рисунке 2.5.

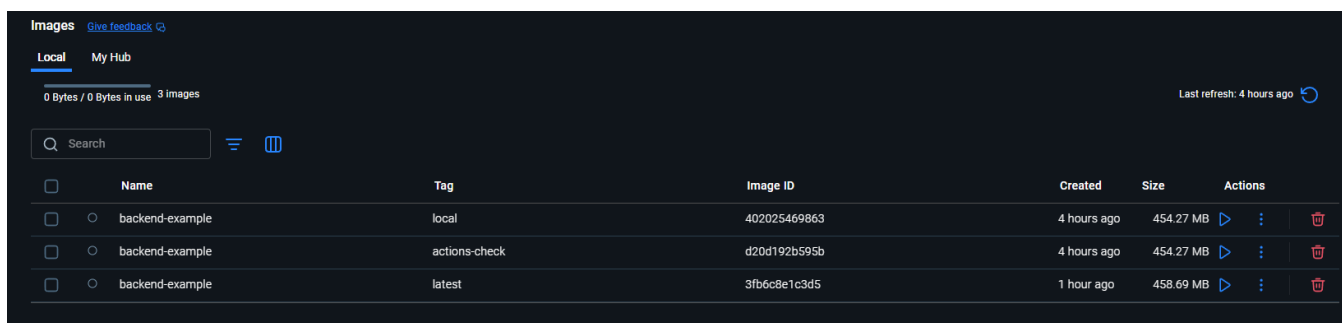


Рисунок 2.5 — Примеры образов

На рисунке 2.6 представлена документация к данному продукту, которая запустилась благодаря Docker.

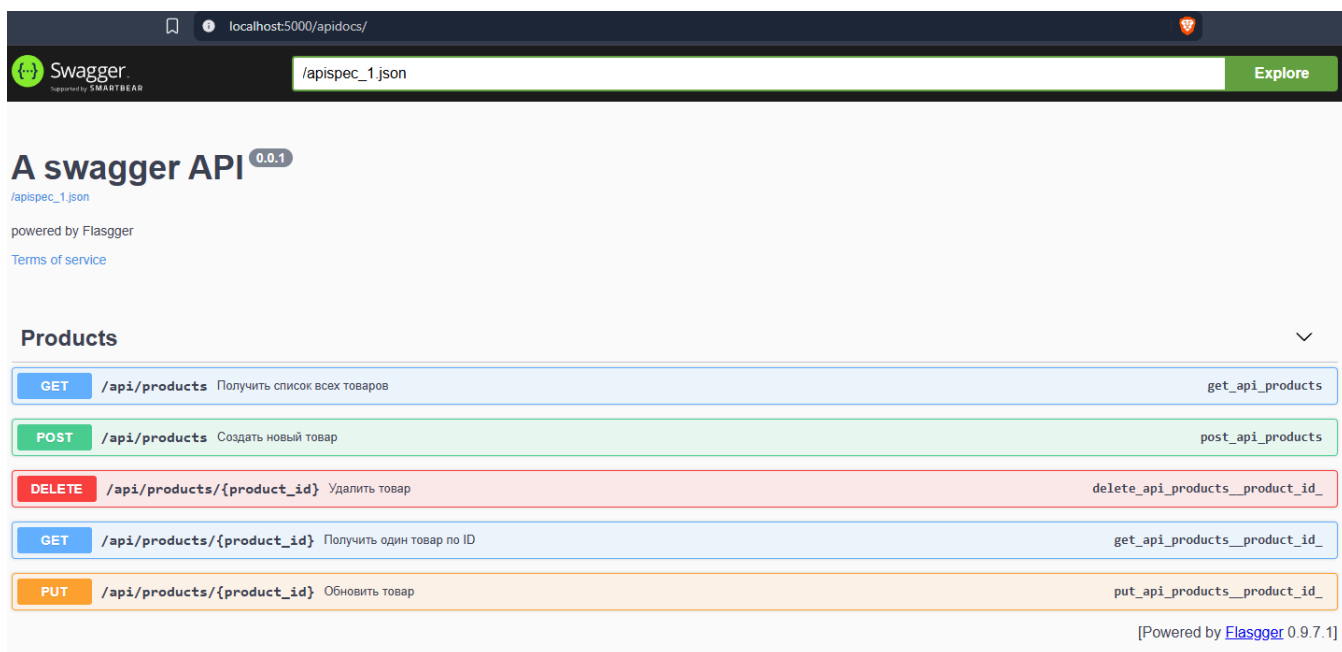


Рисунок 2.6 — Пример документации Swagger

В интерфейсе Docker Desktop мы также можем посмотреть на различные вещи, связанные с контейнером (см. методические рекомендации к Docker).

И так, наш контейнер работает, теперь можно его остановить с помощью команды CTRL + C. Если он продолжает висеть в фоне — docker ps помогает. Или же, можно остановить его по ID контейнера — docker stop <ID_контейнера>.

Теперь перейдем к GitHub и GitHub Actions.

3 build.yml

Так как мы работаем именно с частью GitHub, то мы должны сделать папку для этого. Конечно, перед этим у вас в проекте также должен оказаться файл .gitignore, так как есть те вещи, которые мы не можем отправлять в репозиторий.

На рисунке 3.1 показан скрин того, как может выглядеть данная структура.

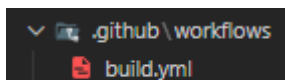


Рисунок 3.1 — Пример структуры

Workflow в GitHub Actions — это прямой аналог pipeline (конвейера) в GitLab CI/CD. Они выполняют одинаковую задачу: автоматизируют сборку, тестирование и деплой кода. И на примере мы увидим, что это действительно так.

Как только мы отправим наши изменения в репозиторий — все сработает как нужно и будут выполнены все те вещи, которые должны произойти. Рассмотрим файл `build.yml` подробнее на рисунках 3.2 и 3.3.

```
.github > workflows > build.yml > ...  
...  
1  name: Test and Build Docker Image  
2  
...  
3  on:  
...  
4  push:  
5    branches: ["main"]  
6  pull_request:  
7  
...  
8  jobs:  
...  
9  test:  
10   runs-on: ubuntu-latest  
11  
...  
12   permissions:  
13     contents: read  
14  
15   steps:  
...  
16     - name: Checkout code  
17       uses: actions/checkout@v4  
18  
...  
19     - name: Set up Python  
20       uses: actions/setup-python@v5  
...  
21       with:  
22         python-version: "3.12"  
23         cache: pip  
24  
...  
25     - name: Install dependencies  
26       run: python -m pip install --upgrade pip && pip install -r requirements.txt  
27  
...  
28     - name: Run tests  
29       run: pytest  
30
```

Рисунок 3.2 — Пример `build.yml` (часть №1)

```

31 build:
32   needs: test
33   if: ${{ needs.test.result == 'success' && github.event_name == 'push' && github.ref == 'refs/heads/main' }}
34   runs-on: ubuntu-latest
35
36   You, 2 hours ago | 1 author (You)
37   permissions:
38     contents: read
39     packages: write
40
41   steps:
42     You, 2 hours ago | 1 author (You)
43     - name: Checkout code
44       uses: actions/checkout@v4
45
46     You, 2 hours ago | 1 author (You)
47     - name: Log in to GitHub Container Registry
48       uses: docker/login-action@v3
49
50     You, 2 hours ago | 1 author (You)
51     with:
52       registry: ghcr.io
53       username: ${{ github.actor }}
54       password: ${{ secrets.GITHUB_TOKEN }}
55
56     You, 5 minutes ago | 1 author (You)
57     - name: Build and push Docker image
58       uses: docker/build-push-action@v5
59
60     You, 5 minutes ago | 1 author (You)
61     with:
62       context: .
63       push: true
64       tags: |
65         ghcr.io/${{ github.repository }}:latest
66         ghcr.io/${{ github.repository }}:${{ github.sha }}

```

Рисунок 3.3 — Пример build.yml (часть №2)

Этот файл представляет собой конфигурацию GitHub Actions — инструмента для автоматизации тестирования и сборки (CI/CD). Скрипт выполняет две основные задачи: запускает тесты на Python при любом изменении кода и, если тесты прошли успешно, собирает Docker-образ и отправляет его в хранилище GitHub (GHCR).

Разберем каждую его часть:

- имя и триггеры: `name` и `on` отвечают за имя сценария (workflow) и за то, когда именно он будет запускаться — в данном случае на `push` в репозиторий и на любой `pull_request`;
- первая задача (работа) — это тестирование (тесты были написаны с помощью `pytest` в отдельном месте):

- 1 `test` — первая независимая задача, которая запускается на виртуальной машине с последней версией Ubuntu;

- 2 `permissions` — этой задаче разрешено только читать содержимое репозитория; `actions/checkout@v4` — это официальный базовый экшен для платформы GitHub Actions, который клонирует ваш репозиторий на виртуальный сервер (раннер), чтобы рабочий процесс (workflow) получил доступ к исходному коду;

- 3 далее — устанавливает Python 3.12 и включает кэширование менеджера пакетов `pip` (кэш нужен для того, чтобы повторные запуски не скачивали библиотеки заново и работали быстрее); `actions/setup-python@v5` — это официальное действие (action) для платформы GitHub Actions, которое устанавливает нужную версию Python или PyPy в рабочее окружение сборки;

- 4 дальше — обновляет `pip` и устанавливает все зависимости вашего проекта, прописанные в файле `requirements.txt`;

- 5 и после — запускает тесты с помощью фреймворка `pytest`. Если какой-то тест упадет, вся задача `test` (и весь процесс) завершится ошибкой;

- вторая задача (работа) — это сборка Docker-образа:

- 1 `needs: test`: задача `build` не начнется, пока успешно не завершится задача `test`;

- 2 `if`: жесткое условие для сборки: сборка и публикация Docker-образа произойдет только если: тесты прошли успешно (`needs.test.result == 'success'`); это был именно `push`, а не `pull_request` (`github.event_name == 'push'`); изменения попали именно в ветку `main` (`github.ref == 'refs/heads/main'`);

- 3 тут в `permissions` права расширены — помимо чтения кода, разрешена запись пакетов (`packages: write`), так как скрипт будет загружать Docker-образ в реестр GitHub;

4 снова скачивает код (уже на новую виртуальную машину для сборки);

5 далее — авторизуется в GitHub Container Registry (GHCR); использует логин автора коммита (github.actor) и автоматический временный пароль/токен (GITHUB_TOKEN), который генерирует сам GitHub;

6 docker/build-push-action@v5 — это официальное действие (action) для GitHub Actions, предназначенное для сборки и публикации Docker-образов с помощью Docker Buildx;

7 и собирает Docker-образ на основе Dockerfile в корне проекта (context: .) и отправляет его в сеть (push: true): образ публикуется сразу под двумя тегами: :latest — всегда указывает на самую свежую версию из ветки main; :\${{ github.sha }} — уникальный тег, равный хэшу коммита Git (чтобы всегда можно было развернуть конкретную старую версию).

Вообще, файл и их количество можно организовать по-разному. Тут могут быть и другие работы, которые необходимы для вашей ситуации. Их можно изучить больше в документации.

Теперь нужно сделать следующее: создать репозиторий и отправить туда наш проект.

4 Создание репозитория и фиксирование наших изменений

Репозиторий создаем как нам привычнее. Лучший способ — зайти на GitHub и создать его вручную, как показано на рисунке 4.1.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * / Repository name *

ttttyuuu / new-repo

new-repo is available.

Great repository names are short and memorable. How about [miniature-barnacle](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository Private

Add README READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore .gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license Licenses explain how others can use your code. [About licenses](#) No license

Create repository

Рисунок 4.1 — Пример создания репозитория

Конечно, потом мы делаем все как привычно через терминал: инициализируем репозиторий, подключаемся через remote к репозиторию на платформе, переходим на нужную ветку, фиксируем изменения и пушим их. Т.е. по-сути точно такой же алгоритм как при обычной работе с GitHub. Только теперь автоматически запускаются Actions.

После пуша репозиторий может выглядеть так, как показано на рисунке 4.2.

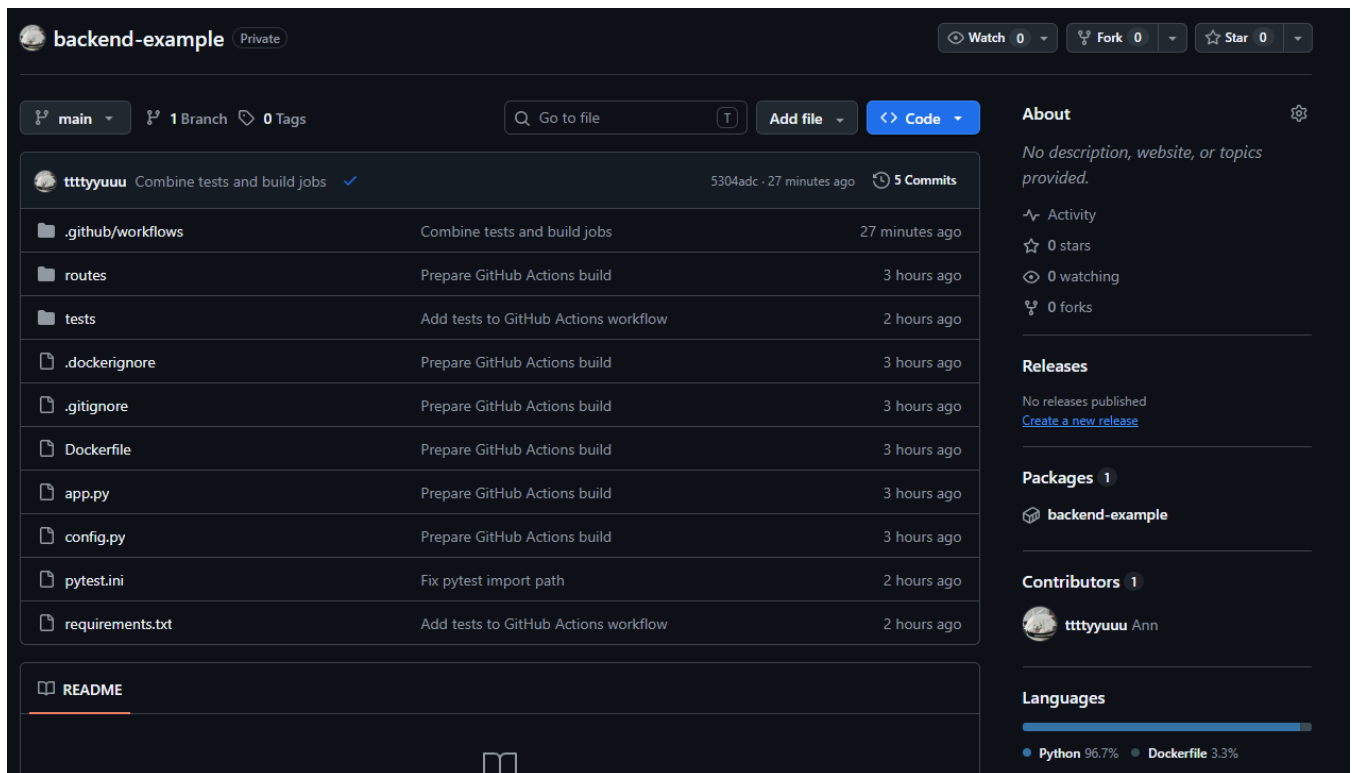


Рисунок 4.2 — Пример внешнего вида репозитория

5 Работа с GitHub Actions и не только

Как можно заметить вверху надпись со знаком — статус нашего сценария. В данном случае он выполнен. Также, здесь у нас есть контейнер в разделе Packages. Посмотрим внутрь него на рисунке 5.1.

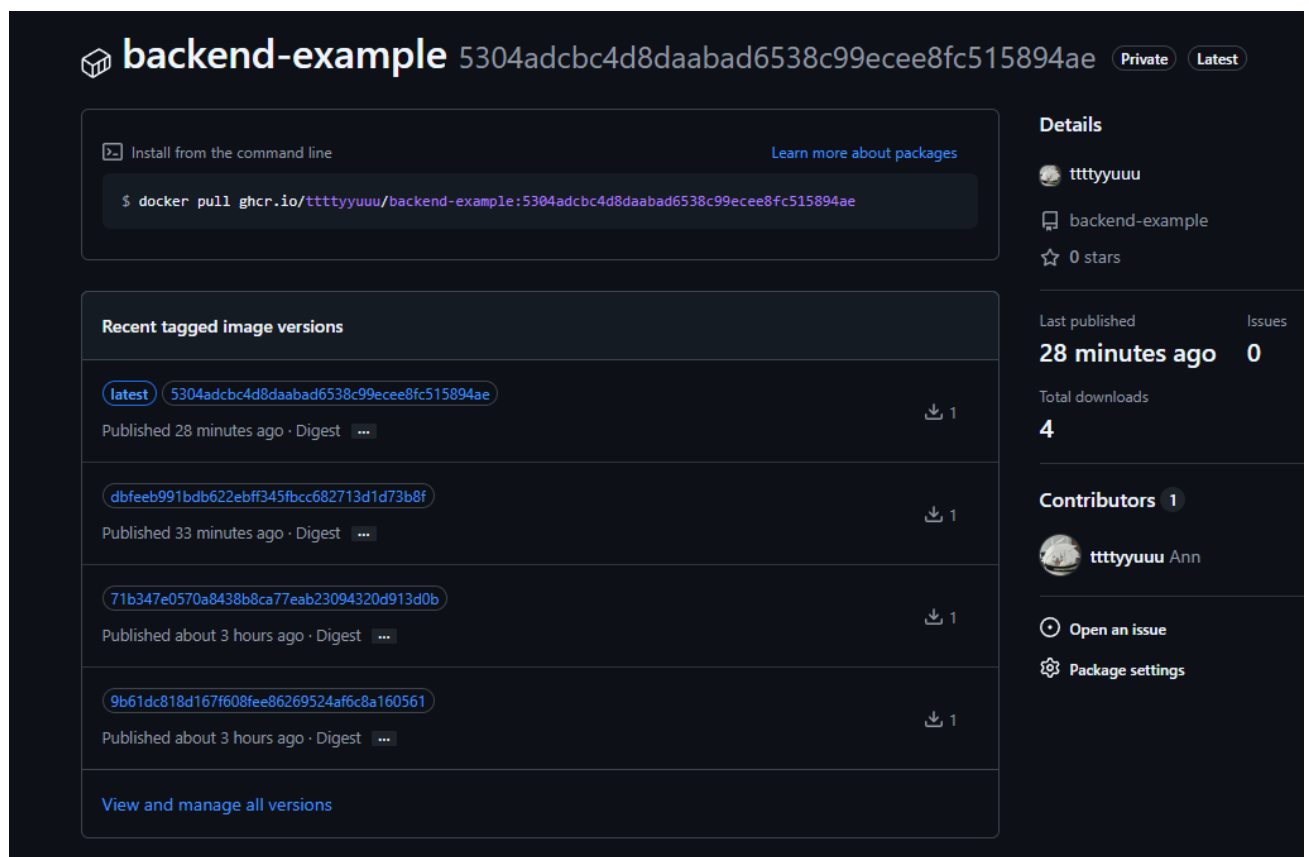


Рисунок 5.1 — Пример внешнего вида страницы образа

Здесь показаны различные версии образа, также показана самая последняя. По разным изменениям можно перейти и найти образ, который относится к данной стадии.

Теперь заглянем во вкладку Actions. На рисунке 5.2 изображена именно она.

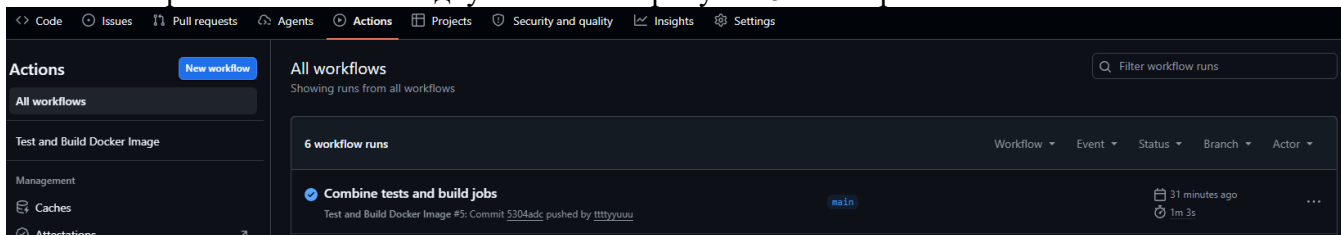


Рисунок 5.2 — Actions

Здесь можно увидеть запуски различных сценариев и как они были выполнены (т.е. отчетность по ним). Посмотрим на сценарий, который нас интересует на рисунке 5.3.

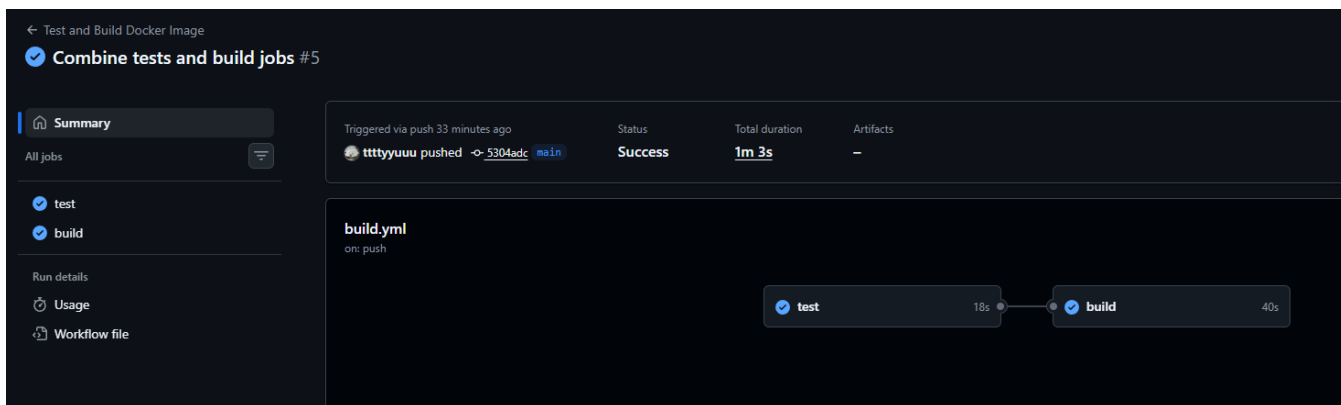


Рисунок 5.3 — Сценарий тестов и сборки

Как можно заметить, обе работы (сценария) прошли успешно. Также, на рисунке 5.4 можно увидеть шаги одной из работ и как именно они выполнялись.

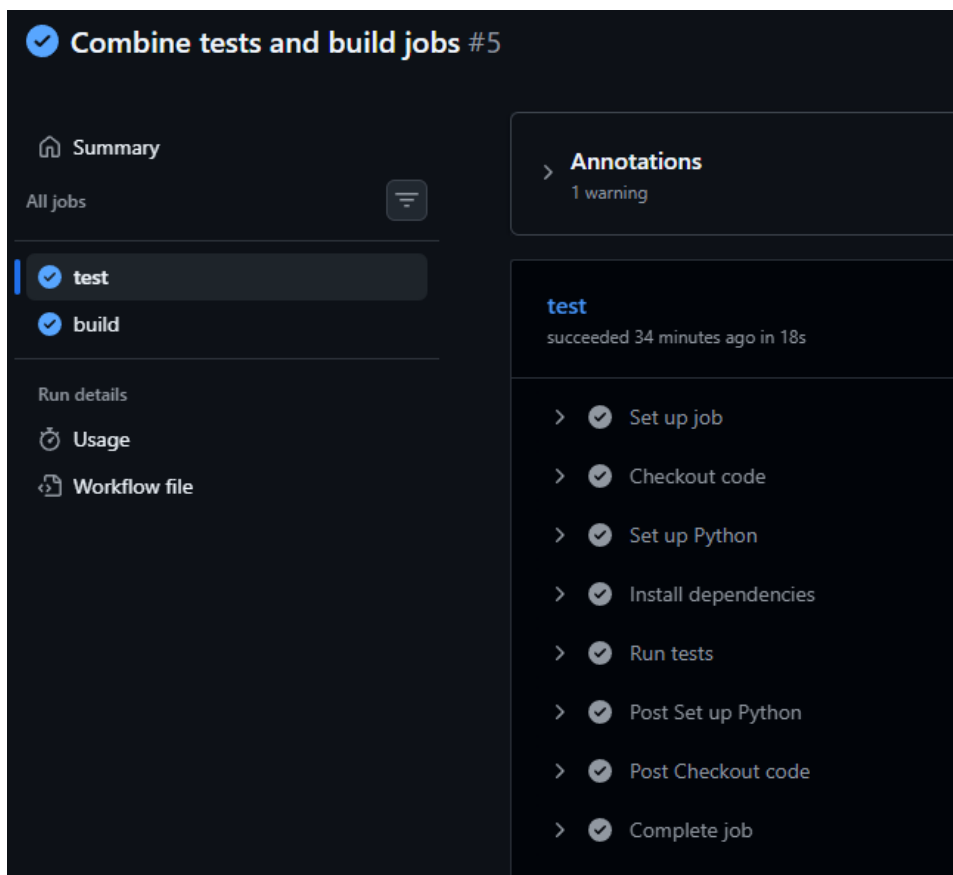


Рисунок 5.4 — Шаги задачи test

Если какая-то задача провалилась — всегда можно посмотреть, на каком шаге это произошло и по какой причине. GitHub Actions имеет систему логгирования. На рисунке 5.5 показан пример.

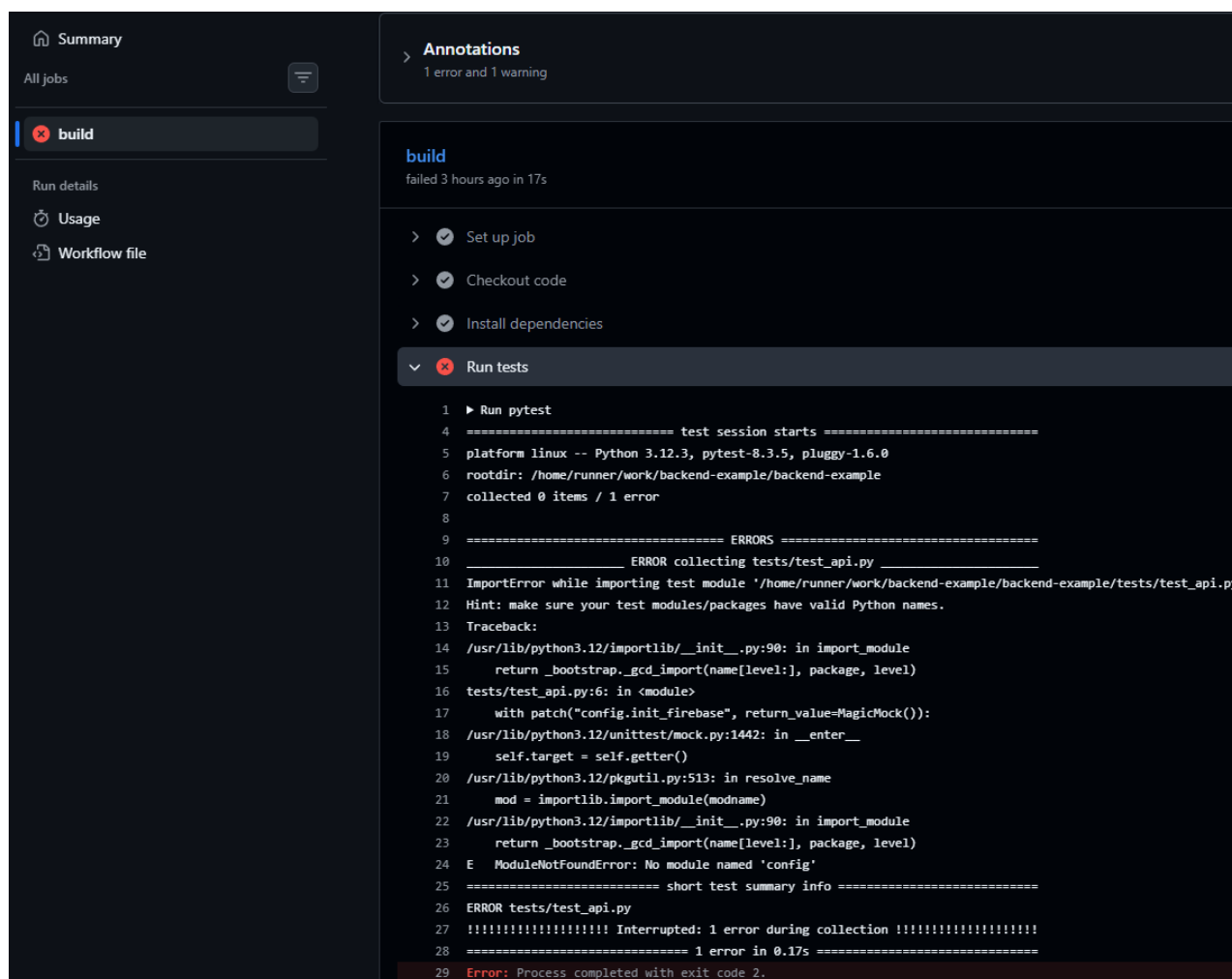


Рисунок 5.5 — Пример ошибки в сценарии

В данном инструменте, так же как и в GitLab, можно хранить секреты (т.е. пароли и ключи к базам данных и другим важным сервисам, которые привязаны к вашему приложению (если, к примеру, мы тестируете какие-то запросы или еще что-то с помощью задач).

На рисунках 5.6 и 5.7 ниже изображены разделы, по которым можно добраться до них в настройках проекта.

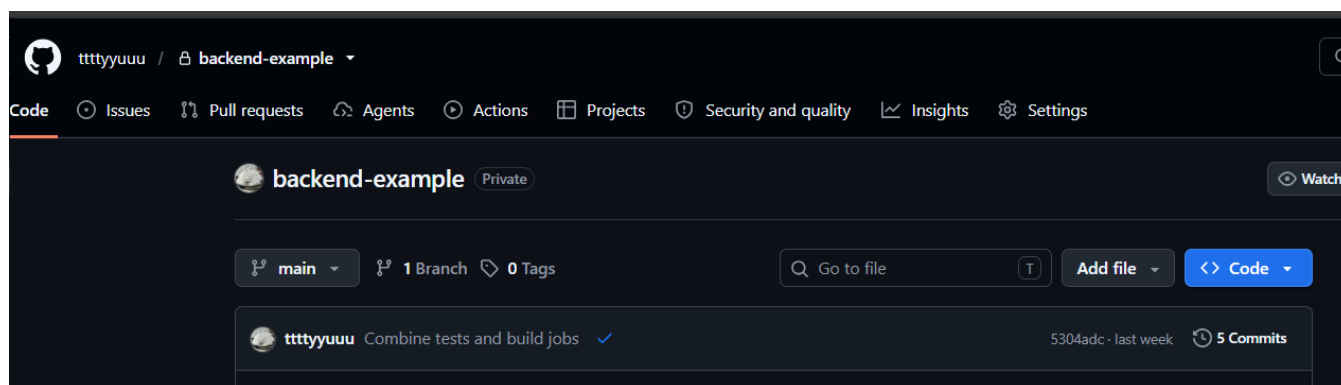


Рисунок 5.6 — Верхняя панель проекта (настройки)

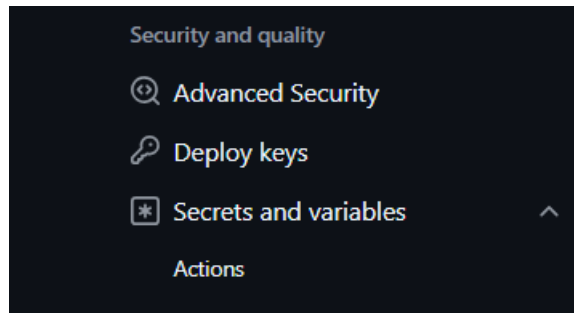


Рисунок 5.7 — Раздел левого меню настроек, которое сделано для хранения секретов

Чтобы перейти в создание секретов именно для этого проекта, нужно перейти в раскрывающийся список **Secrets and variables** и в **Actions**. После чего вам откроется окно, представленное на рисунке 5.8.

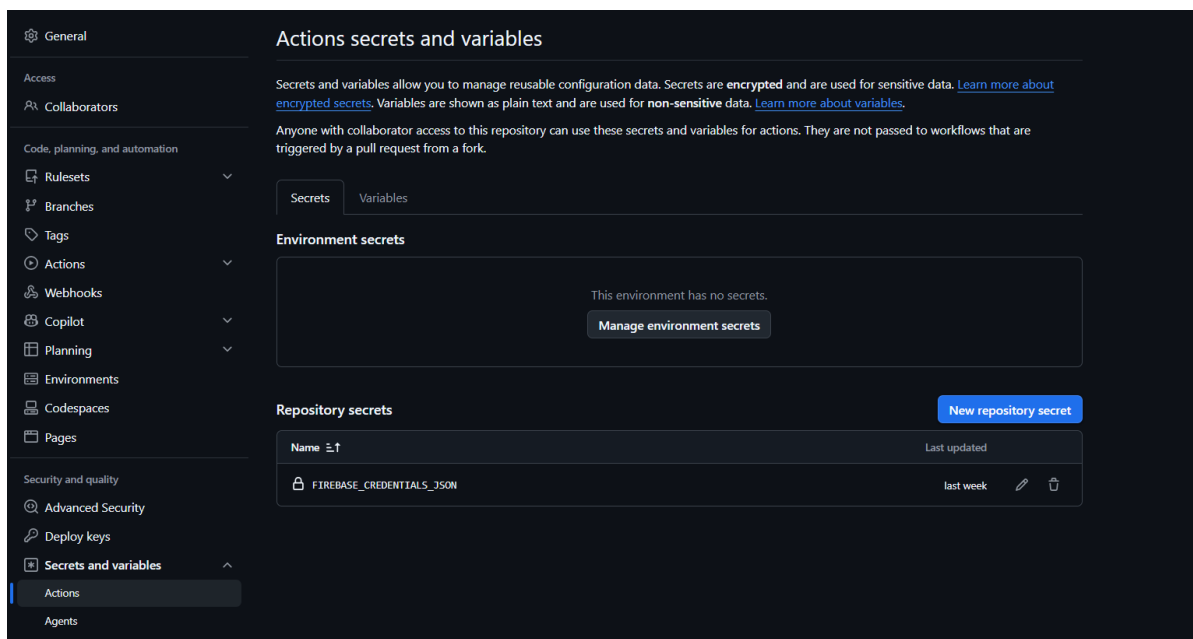


Рисунок 5.8 — Action secrets and variables

Как можно заметить по рисунку, в демонстрационном проекте хранится ключ от Firebase. Ключи и другие данные в этом разделе не хранятся в сыром виде, они шифруются — так что никто не сможет просто взять и скопировать данные из этого раздела. Можно только обновить их, а самое поле будет пустым. На рисунке 5.9 показано, как это будет выглядеть.

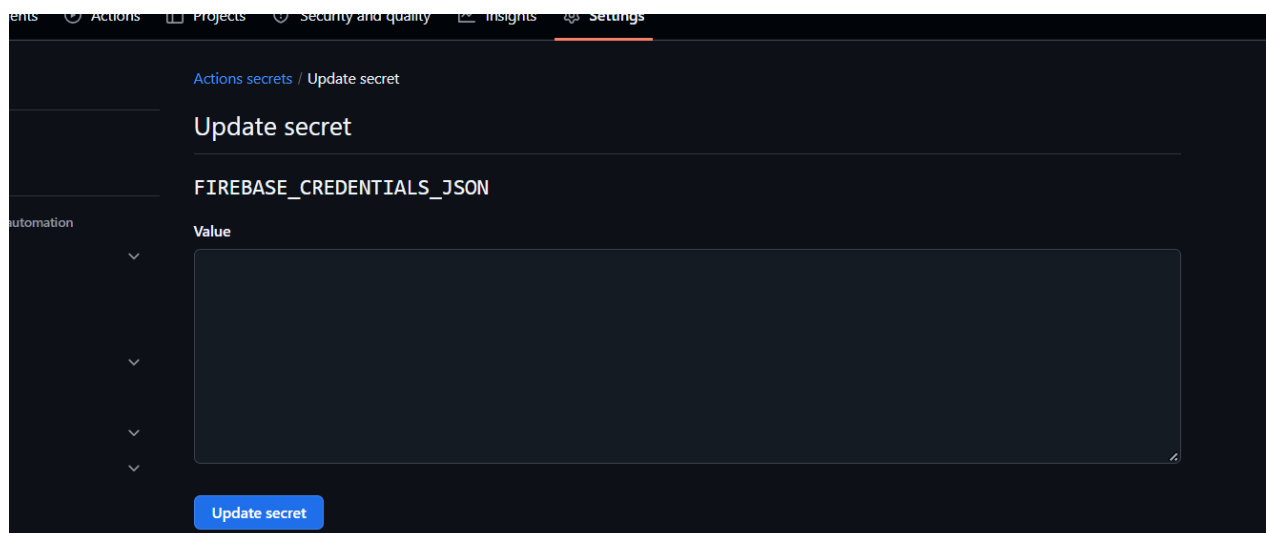


Рисунок 5.9 — Изменение значения

Теперь задание, связанное с Pull Requests. Попробуйте найти в Интернете, как именно с ними работать, как их создавать и какие команды для этого использовать, и опишите данный процесс с отчете, а также, прикрепите скриншоты вашей работы с данной функцией.

7 Отчет к работе

Отчетом к практическому занятию является выполнение всех шагов из практической работы, прикрепление ссылки на репозиторий (публичный) и добавление скриншотов работы с Docker и GitHub Actions.

Заключение

В ходе данного практического занятия мы освоили основные принципы контейнеризации программного проекта с использованием Docker, а также настроили автоматизацию его тестирования и сборки посредством GitHub Actions с публикацией Docker-образа в GitHub Container Registry.